

Processing Structured Records

Loop through realistic data one dictionary at a time.

List of dictionaries

```
records = [  
    {"record_id": "EQ-1001", "status": "active"},  
    {"record_id": "EQ-1002", "status": "inactive"},  
]
```

Today's mission

Summarize, classify, update, and collect records using loops.

Input data → record-by-record logic → output collections

Lists of dictionaries

A dataset is often a collection of structured records.

```
records = [  
    {  
        "record_id": "EQ-1001",  
        "status": "active",  
        "error_count": 0,  
    },  
    {  
        "record_id": "EQ-1002",  
        "status": "active",  
        "error_count": 2,  
    },  
]
```

Outer structure: list

Each item: dictionary

**Each dictionary: one
record**

Each key: a field name

This resembles rows in a table,
JSON objects, API responses, and
CSV data after parsing.

BLOCK 22

Record-by-record processing

The loop takes one dictionary at a time.

```
for record in records:  
    print(record["record_id"])
```



Same code. Different current record.

BLOCK 22

Accessing fields inside a loop

Pull values out of the current dictionary.

```
for record in records:  
    record_id = record["record_id"]  
    status = record["status"]  
    error_count = record["error_count"]  
  
    print(record_id, status, error_count)
```

Why assign fields to variables?

- Improves readability
- Avoids repeated dictionary lookups in the code
- Makes the report line easier to understand

Think: “unpack the record into useful names.”

Conditional logic inside loops

Every record can take a different path.

```
for record in records:
    if record["error_count"] == 0:
        print(f'{record["record_id"]}: clean')
    else:
        print(f'{record["record_id"]}: has errors')
```

Loop

Visit every record

If

Classify this record

Pattern: for each record, apply the same decision rule.

Updating records

A loop can add a new field to every dictionary.

```
for record in records:
    if record["error_count"] == 0:
        record["quality_status"] = "valid"
    else:
        record["quality_status"] = "review"
```

Before

```
{"record_id": "EQ-1002",
 "error_count": 2}
```

After

```
{"record_id": "EQ-1002",
 "error_count": 2,
 "quality_status": "review"}
```

The original dictionaries are changed.

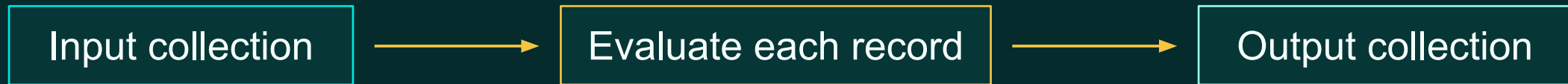
BLOCK 22

Collecting matching records

Build a new list from records that pass a test.

```
valid_records = []  
  
for record in records:  
    if record["error_count"] == 0:  
        valid_records.append(record)
```

**This is the core filter
pattern.**



Collecting rejected records

The same pattern can capture records needing attention.

```
rejected_records = []  
  
for record in records:  
    if record["error_count"] > 5:  
        rejected_records.append(record)
```

Why collect them?

- Generate a report
- Send to a review queue
- Write to a separate file later
- Explain what failed

Rejected records are still data.

BLOCK 22

Three output collections

Every record should land in exactly one bucket.

```
ready_records = []  
warning_records = []  
rejected_records = []
```

ready

warning

rejected

Classification logic decides the
bucket.

Exactly one output collection keeps
the summary clean.

Building summary counts

Counters describe the dataset while you loop.

```
active_count = 0
inactive_count = 0
records_with_errors = 0
total_errors = 0

for record in records:
    if record["status"] == "active":
        active_count += 1
    else:
        inactive_count += 1

    if record["error_count"] > 0:
        records_with_errors += 1
        total_errors += record["error_count"]
```

One loop can produce multiple summaries.

Separate if statements allow multiple updates for the same record.

Use if/elif when choosing one category.

BLOCK 22

Using enumerate()

Sometimes you need the position and the record.

```
for index, record in enumerate(records):  
    print(index, record["record_id"])
```

index starts at 0

```
for number, record in enumerate(records, start=1):  
    print(f'{number}. {record["record_id"]}')"
```

start=1 is nicer for people

Use enumerate() for numbered reports or troubleshooting.

BLOCK 22

Iterating through a dictionary

Loop through field names or field/value pairs.

```
record = {  
    "record_id": "EQ-1001",  
    "status": "active",  
    "error_count": 0,  
}
```

```
for key in record:  
    print(key)
```

```
for key, value in record.items():  
    print(key, value)
```

Useful when fields need to be inspected dynamically.

BLOCK 22

Guided dataset

We will process four structured records.

```
records = [  
    {"record_id": "EQ-1001", "status": "active",  
     "error_count": 0, "completion": 100},  
    {"record_id": "EQ-1002", "status": "active",  
     "error_count": 2, "completion": 95},  
    {"record_id": "EQ-1003", "status": "inactive",  
     "error_count": 0, "completion": 100},  
    {"record_id": "EQ-1004", "status": "active",  
     "error_count": 8, "completion": 76},  
]
```

Fields

record_id

status

error_count

completion

**Exercise time: 3 minutes to
copy the data.**

BLOCK 22

Guided exercise: summary counts

Initialize counters before the loop.

```
record_count = 0
active_count = 0
inactive_count = 0
records_with_errors = 0
total_errors = 0
complete_count = 0
review_count = 0
```

Calculate

- Number of records
- Number active
- Number inactive
- Number with errors
- Total errors
- Number complete
- Number requiring review

Exercise time: 8 minutes

BLOCK 22

Guided exercise: add processing_status

Each record gets a new classification field.

```
for record in records:
    if record["status"] != "active":
        record["processing_status"] = "rejected"
    elif record["error_count"] > 5:
        record["processing_status"] = "rejected"
    elif record["error_count"] > 0:
        record["processing_status"] = "warning"
    elif record["completion"] < 100:
        record["processing_status"] = "warning"
    else:
        record["processing_status"] = "ready"
```

Order matters

Rejected checks first

Warnings next

Ready only if nothing
failed

Exercise time: 8 minutes

BLOCK 22

Guided exercise: formatted report

Use `enumerate()` to make output readable.

```
for number, record in enumerate(records, start=1):  
    print(f"{number}. {record['record_id']}")  
    print(f"    Status: {record['status']}")  
    print(f"    Errors: {record['error_count']}")  
    print(f"    Processing: {record['processing_status']}")
```

**Output should help
humans understand the
data.**

Readable reporting is part of
programming.

Exercise time: 5 minutes

BLOCK 22

Applied challenge: split output collections

Route each record into exactly one list.

```
ready_records = []  
warning_records = []  
rejected_records = []  
  
for record in records:  
    # decide the status  
    # append to exactly one collection
```

At the end, display:

- Total input records
- Ready count
- Warning count
- Rejected count
- Percentage ready
- IDs of rejected records

Challenge time: 15 minutes

Challenge scaffold: classification first

Classify the record, then route it.

```
for record in records:
    if record["status"] != "active":
        destination = "rejected"
    elif record["error_count"] > 5:
        destination = "rejected"
    elif record["error_count"] > 0:
        destination = "warning"
    elif record["completion"] < 100:
        destination = "warning"
    else:
        destination = "ready"

    record["processing_status"] = destination
```

Why use destination?

It separates deciding from appending.

Cleaner code is easier to debug.

Challenge scaffold: route records

Append the current record to the right output list.

```
if destination == "ready":  
    ready_records.append(record)  
elif destination == "warning":  
    warning_records.append(record)  
else:  
    rejected_records.append(record)
```

**Exactly one append
per record.**

If a record lands in two
places, the logic is wrong.

If a record lands nowhere,
the logic is incomplete.

Challenge scaffold: final summary

Turn collections into useful reporting.

```
total_records = len(records)
ready_count = len(ready_records)
warning_count = len(warning_records)
rejected_count = len(rejected_records)

percentage_ready = ready_count / total_records * 100

print(f"Ready: {ready_count}")
print(f"Warning: {warning_count}")
print(f"Rejected: {rejected_count}")
print(f"Ready %: {percentage_ready:.1f}%")
```

Rejected IDs

```
for record in rejected_records:
    print(record["record_id"])
```

Use collections to summarize collections.

BLOCK 22

Common mistakes

Structured records add a few new traps.

Wrong key name

```
record["errors"]  
# but key is error_count
```

Updating outside loop

```
for record in records:  
    pass  
record["status"] = "ready"
```

Appending wrong thing

```
ready_records.append(  
    record["record_id"]  
)
```

Debug strategy

Print one record. Print one field. Print one decision.

BLOCK 22

Knowledge check and bridge

You now have the core data-processing pattern.



Students should be able to explain:

- Why the outer structure is a list
- Why each record is a dictionary
- How to retrieve a field inside a loop
- How to update records
- How to collect matching records

Next up: more automation patterns over datasets.